Information Processing Letters 39 (1991) 269–276
North-Holland

13 September 1991

Smoothsort revisited

Coenraad Bron and Wim H. Hesselink

Department of Computing Science, Rijksuniversiteit Groningen, P.O. Box 800, 9700 AV Groningen, Netherlands

Communicated by F.B. Schneider
Received 8 April 1991
Revised 13 May 1991

Abstract

Bron, C. and W.H. Hesselink, Smoothsort revisited, Information Processing Letters 39 (1991) 269–276.

A renewed analysis of Dijkstra's array sorting algorithm Smoothsort showed that the removal of a complexity improved the performance. The resulting algorithm has a worst-case behaviour of order $N \cdot \log N$. In the average case, it has more or less the same speed as heapsort. For arrays that are initially almost sorted, it is much faster than heapsort and can compete with quicksort. The correctness is argued by means of invariants. The performance is measured in terms of comparisons and exchanges, both in the analysis of worst-case behaviour and in the experimental data.

Keywords: Analysis of algorithms, design of algorithms, performance evaluation, sorting

Introduction

In search for algorithms suitable for a software library, we considered Dijkstra's algorithm smoothsort, cf. [1], and compared it experimentally with the well-known algorithms quicksort and heapsort (see, e.g., [2]). It turned out that, in the average case, smoothsort is noticeably slower than heapsort.

We then modified smoothsort by weakening its invariant. In this way, we got a simpler algorithm with a better performance in the average case and in the worst case. Since the algorithm is based on a row of trees of decreasing sizes (such as can be seen in the Dutch polders) we propose the name poplar sort. Although it is a variation of smoothsort, the name smoothsort would not be correct. For the main characteristic of smoothsort is that it is of order N if the array is initially sorted, whereas in that case poplar sort still requires order $N \cdot \log N$ comparisons. Two other modifications are that poplar sort uses a small array instead of a bit pattern and perfect binary trees instead of the "Leonardo trees" of [1].

Actually, in [1], both the weaker invariant and the perfect binary trees are mentioned but rejected. So the algorithm is not new. In our investigation, however, an algorithm emerged that was simpler to present and had better average behaviour than the original version of smoothsort thanks to the fact that the order N behaviour of smoothsort for initially almost sorted sequences has been sacrificed. In view of these two aspects we consider the version described here worthy of wider attention. It can also be regarded as a stepping stone for understanding smoothsort.

1. Development of poplar sort

Since poplar sort is a simplification of smoothsort, cf. [1], and smoothsort is not well known, we develop the algorithm from scratch. For simplicity, we describe sorting the array a declared by

```
var a: array[1..N] of integer.
```

0020-0190/91/$03.50 © 1991 – Elsevier Science Publishers B.V. All rights reserved

In this note the only command that modifies the array will be the command *swap*(p, q) that exchanges the values $a[p]$ and $a[q]$, provided p and q are in the index domain $[1..N]$. This implies that the array remains a permutation of the initial array.

Just as heapsort and smoothsort, poplar sort is a selection sort in which the unsorted part of the array is structured in such a way that the element to be added to the sorted part is easily located. We introduce a variable k to separate the two parts of the array. The invariant of selection sort is

$$P0: \quad (\forall i, j: 1 \leqslant i < j \wedge k < j \leqslant N: a[i] \leqslant a[j]).$$

Clearly, $P0 \wedge k \leqslant 1$ implies that the array is sorted.

The unsorted part of the array is organised as a juxtaposition of consecutive poplars (trees of a special type). Let us first define a heap to be a binary tree having its maximal element in the root and having two subtrees each of which is empty or a heap. A heap is called perfect if both subtrees are empty or perfect heaps of the same size. A poplar is defined to be a perfect heap mapped on a contiguous section of the array in the form of two subpoplars (or empty sections) followed by the root. This concept is formalised by the predicate

$$\begin{aligned} popl(p, q) \equiv\ & 0 \leqslant p < q \leqslant N \wedge odd.(q-p) \\ & \wedge (p+1 < q \Rightarrow a[p \ominus q] \leqslant a[q] \wedge a[q-1] \leqslant a[q] \\ & \wedge popl(p, p \oplus q) \wedge popl(p \oplus q, q-1)), \end{aligned}$$

where $p \oplus q$ is an abbreviation of the intermediate number

$$p \oplus q = (p+q) \ \mathbf{div}\ 2.$$

Now $popl(p, q)$ means that the segment $[p+1..q]$ contains a poplar. By induction on the size of the tree, we have that $popl(p, q)$ implies $a[i] \leqslant a[q]$ for all indices i with $p < i \leqslant q$. For the purpose of locating the largest element of the unsorted part, it suffices to consider the roots of the poplars only.

We introduce an integer variable t for the number of poplars and an array r to keep the positions of the roots of the poplars:

```
var t: integer;
var r: array [0..L] of integer;
```

with the invariant

$$\begin{aligned} P1: \quad & 0 \leqslant t \leqslant L \wedge r[0] = 0 \wedge r[t] = k \wedge 0 \leqslant k \leqslant N \\ & \wedge (\forall i: 0 < i \leqslant t: popl(r[i-1], r[i])). \end{aligned}$$

To keep the number of poplars as small as possible, we organise the poplars in descending sizes, always combining two poplars of equal size into a poplar of the next larger size with a possible exception at the end of the row. This is formalised in the invariant

$$\begin{aligned} P2: \quad & (\forall i: 1 < i \leqslant t: \\ & r[i] - r[i-1] < r[i-1] - r[i-2] \\ & \vee (i = t \wedge r[t] - r[t-1] = r[t-1] - r[t-2])). \end{aligned}$$

Now the global structure of the algorithm is

(0) $k := 0;\ t := 0;\ r[0] := 0;$
while $k \neq N$ **do** {invariant $P1 \wedge P2$}
$S0;\ k := k + 1$ **od**;
$\{P1 \wedge P2 \wedge k = N$, which implies $P0\}$
while $k > 1$ **do** {invariant $P0 \wedge P1 \wedge P2$}
$S1;\ k := k - 1$ **od**
$\{P0 \wedge k \leqslant 1$, so array a is sorted$\}$,

where commands $S0$ and $S1$ do not modify k.

We now investigate the maximal number of poplars. From the definition, the size of a poplar is always one less than a power of two:

(1) $popl(p, q) \Rightarrow (\exists d: d \geqslant 1: q - p = 2^d - 1)$.

If $t > 0$, the conjunction $P1 \wedge P2$ implies

$$
\begin{aligned}
& N \\
\geqslant\ & \{P1\} \\
& r[t] - r[0] \\
=\ & \{\text{arithmetic}\} \\
& \left(\sum i: 0 < i \leqslant t: r[i] - r[i-1]\right) \\
\geqslant\ & \{P2 \text{ and } (1);\ t > 0\} \\
& \left(\sum i: 0 < i < t: 2^{t-i} - 1\right) + 1 \\
=\ & \{\text{arithmetic}\} \\
& 2^t - t \\
\geqslant\ & \{t \leqslant 2^{t-1} \text{ since } t \text{ is an integer}\} \\
& 2^{t-1}.
\end{aligned}
$$

This proves $2^{t-1} \leqslant N$. In this way, we get

(2) $P1 \wedge P2 \Rightarrow t \leqslant 1 + \lfloor log\ N \rfloor$,

where the logarithms have base 2. We can therefore choose constant L to be

(3) $L = 1 + \lfloor log\ N \rfloor$.

In order to create poplars and to restore poplars of which the root has been replaced by a value that need not be maximal, we define the slightly weaker concept of semipoplar in which the root value $a[q]$ can be exceptional. More precisely, we define the predicate

$$
\begin{aligned}
spopl(p, q) \equiv\ & 0 \leqslant p < q \leqslant N \wedge odd.(q - p) \\
& \wedge (p + 1 < q \Rightarrow popl(p, p \oplus q) \wedge popl(p \oplus q, q - 1)).
\end{aligned}
$$

Note that this definition is the same as that for *popl* with the comparisons with $a[q]$ removed.

A semipoplar can be transformed into a poplar by means of the procedure *sift* which is a variation of the procedure *sift* in heapsort:

```
procedure sift(const p,q: integer)
{pre spopl(p, q); post popl(p, q);
 only permuting array a in [p + 1..q]};
   |[if q − p > 1 then
        x := q;
        if a[q − 1] > a[x] then x := q − 1 fi;
        if a[p ⊕ q] > a[x] then x := p ⊕ q fi;
        if x ≠ q then
          swap(x, q);
          if x = q − 1 then sift(p ⊕ q, x) else sift(p, x) fi fi
    fi ]|.
```

The proof of the correctness of this recursive procedure may be left to the reader.

For the construction of command $S0$, we observe that invariant $P1$ allows the creation of a new poplar of size 1. If the last two poplars of the row have equal sizes, however, the addition of an extra poplar is forbidden by invariant $P2$. Therefore, in the latter case, these two poplars are joined with a new root and then procedure *sift* is applied. This leads to the refinement

```
S0 = |[  {P1 ∧ P2 ∧ k ≠ N}
     if t ⩾ 2 cand r[t] − r[t − 1] = r[t − 1] − r[t − 2] then
       t := t − 1; r[t] := k + 1;
         {spopl(r[t − 1], r[t])}
       sift(r[t − 1], r[t])
     else t := t + 1; r[t] := k + 1 fi ]|.
```

In this way, the first phase of the algorithm establishes the invariant $P0 \wedge P1 \wedge P2$.

We now come to the sorting phase. Here we first strengthen the invariant $P0 \wedge P1 \wedge P2$ with an additional conjunct

$$Q: \quad (\forall i: 0 < i \leqslant r[t]: a[i] \leqslant a[r[t]]),$$

which states that $a[r[t]]$ is the maximal value of the unsorted part of the array. This conjunct is established by a search for the number m of the root with the maximal value, if necessary followed by an interchange of the roots numbered t and m and a restoration of the semipoplar with root $r[m]$. This is coded in

```
procedure relocate
{const k, r, t; pre P0 ∧ P1 ∧ P2 ∧ t ⩾ 1;
 post P0 ∧ P1 ∧ P2 ∧ Q;
 only permuting array a in [1..r[t]]};
   var m, j: integer;
   |[m := t;
   for j := 1 to t − 1 do
     if a[r[m]] < a[r[j]] then m := j fi od;
   if m ≠ t then
     swap(r[m], r[t]);
     sift(r[m − 1], r[m]) fi ]|.
```

It is here that poplar sort deviates from smoothsort. For the invariant of smoothsort contains the additional conjunct

$$(\forall i, j: 0 < i \leqslant j \leqslant t: a[r[i]] \leqslant a[r[j]]),$$

which implies predicate Q. In the average case the preservation of this conjunct is more costly than procedure *relocate*.

Procedure *relocate* establishes $P0 \wedge Q$, which predicate is easily seen to satisfy the Hoare triple

$$\{P0 \wedge Q\} \quad k := k-1 \quad \{P0\}.$$

For the preservation of $P1 \wedge P2$, we observe that command $k := k-1$ requires the last poplar to be removed if it is a singleton and to be split otherwise. In this way, we obtain the following refinement of command $S1$ in program (0):

$S1 = |[\{P0 \wedge P1 \wedge P2 \wedge k > 1\}$
relocate;
if $r[t-1] = k-1$ **then** $t := t-1$
else $r[t] := r[t-1] \oplus k$;
$t := t+1$; $r[t] := k-1$ **fi** $]|$.

This concludes the development of the algorithm.

In the actual coding of the algorithm several optimisations are possible, the effect of which depends on compiler characteristics and is usually only of order N. For the discussion of the complexity, the algorithm is summarised.

(4)
$k := 0$; $t := 0$; $r[0] := 0$;
while $k \neq N$ **do** {invariant $P1 \wedge P2$}
if $t \geqslant 2$ **cand** $k + r[t-2] = 2 \cdot r[t-1]$ **then**
$t := t-1$;
A: *sift*$(r[t-1], k+1)$
else $t := t+1$ **fi**;
$k := k+1$; $r[t] := k$ **od**;
while $k > 1$ **do** {invariant $P0 \wedge P1 \wedge P2$}
$m := t$;
B: **for** $j := 1$ **to** $t-1$ **do**
if $a[r[m]] < a[r[j]]$ **then** $m := j$ **fi** **od**;
C: **if** $m \neq t$ **then**
swap$(r[m], k)$;
sift$(r[m-1], r[m])$ **fi**;
if $r[t-1] = k-1$ **then** $t := t-1$
else $r[t] := r[t-1] \oplus k$;
$t := +1$; $r[t] := k-1$ **fi**;
$k := k-1$ **od**.

2. Analysis of complexity

Program (4) consists of an initialisation followed by two repetitions which are essentially for-loops. The operations on t and r are completely independent of the value of array a. We concentrate on the commands labelled A, B, C. In fact, in the other commands array a is not accessed and, as is easily

verified, the other commands take time proportional to N. Commands A, B, C mainly consist of sifts, comparisons and swaps. Since procedure *sift* mainly consists of comparisons and swaps, we take the total amount of comparisons and swaps as our measure of complexity.

Let us say that a semipoplar has depth d if it has $2^d - 1$ elements. So, in particular, the singleton has depth 1. The sift of a semipoplar of depth d requires at most $d-1$ swaps and $2 \cdot (d-1)$ comparisons. If we may assume that all positions in the poplar have equal probability of being the final position of the initial root, the expected number of swaps is

$$\begin{aligned}&\big(\Sigma i\colon 0 \leqslant i < d\colon i \cdot 2^i\big)/(2^d - 1)\\ &= \big((d-2)\cdot 2^d + 2\big)/(2^d - 1)\\ &= d - 2 + \big(d/(2^d - 1)\big).\end{aligned}$$

This implies that the average-case behaviour of sift is not much better than the worst-case behaviour. We therefore concentrate on the worst-case behaviour. Part A of the algorithm represents the use of *sift* for the construction of poplars. Let $f(d)$ be the maximal number of swaps needed for the construction of a poplar of depth d. Then we have $f(1) = 0$ and

$$f(d+1) = d + 2 \cdot f(d).$$

This recurrence relation has the solution $f(d) = 2^d - 1 - d$.

Thus, the construction of a poplar of size m takes fewer than m swaps and fewer than $2 \cdot m$ comparisons. Therefore, the construction in part A of a row of disjoint poplars takes less than N swaps and $2 \cdot N$ comparisons. These upper bounds are rather sharp. In the extremely favourable case where the array is initially sorted, part A requires zero swaps and $2 \cdot (N - \#\text{leaves}) \leqslant N$ comparisons.

For the calculation of the contribution of parts B and C, we assume that $N = 2^{d+1} - 2$. This is the worst case. In fact, the assumption implies that, during execution of the second loop, all configurations of rows of poplars of depth $\leqslant d$ that are permitted by invariant $P2$ will occur.

Each execution of part B requires $\tau(k) - 1$ comparisons, where $\tau(k)$ is the number of poplars that fill a segment of size k. In view of invariant $P2$ and formula (1), we have $\tau(k) = t$ if $k = \Sigma_{j=1}^{t}(2^{e.j} - 1)$, where the depths $e.j$ with $1 \leqslant j \leqslant t$ form a descending sequence that is decreasing with a possible exception at the end. Since $k \leqslant 2^{d+1} - 2$, we have $1 \leqslant e.j \leqslant d$ for all j. The sequence e is completely determined by its range $I = \{e.i \mid i\}$ and the question whether the smallest depth occurs once or twice. We then have $t = \#I$ or $t = \#I + 1$, respectively. Since all numbers k with $0 < k \leqslant N$ occur and $N = 2^{d+1} - 2$, all nonempty subsets I of $[1..d]$ occur with both alternatives. Therefore, the total number of comparisons for B is

$$\begin{aligned}&\big(\textstyle\sum k\colon 0 < k \leqslant N\colon \tau(k) - 1\big)\\ =\ &\{\text{arithmetic}\}\\ &\big(\textstyle\sum k\colon 0 < k \leqslant N\colon \tau(k)\big) - N\\ =\ &\{\text{argument given above}\}\\ &\big(\textstyle\sum I\colon I \subseteq [1..d] \wedge I \neq \emptyset\colon \#I + (\#I + 1)\big) - N\\ =\ &\{\text{the number of sets } I \text{ is } 2^d - 1\text{; add empty set } I\}\\ &2^d - 1 - N + 2 \cdot \big(\textstyle\sum I\colon I \subseteq [1..d]\colon \#I\big)\\ =\ &\{\text{if } J \text{ is the complement of } I\text{, then } \#I + \#J = d\}\end{aligned}$$

$$
\begin{aligned}
& 2^d - 1 - N + \left(\sum I:\ I \subseteq [1..d]:\ d\right) \\
= & \quad \{\text{the number of sets } I \text{ is } 2^d\} \\
& 2^d - 1 - N + d \cdot 2^d \\
= & \quad \{\text{use } N = 2^{d+1} - 2\} \\
& \tfrac{1}{2} \cdot N \cdot (d-1) + d.
\end{aligned}
$$

The worst-case contribution of part C consists of $N-1$ swaps and sifts. In the case that $N = 2^{d+1} - 2$, these $N-1$ sifts require at most M swaps and $2 \cdot M$ comparisons, where

$$
\begin{aligned}
& M \\
& = \left(\sum i:\ 1 \leqslant i \leqslant d:\ (i-1) \cdot 2^i\right) \\
& = (d-2) \cdot 2^{d+1} + 4 \\
& = (d-2) \cdot N + 2 \cdot d.
\end{aligned}
$$

Therefore, asymptotically, part C requires four times as many comparisons as part B, and a corresponding number of swaps. Part B can be optimised, but, in view of the statistics in Section 4, the overhead is only justified if the algorithm is used for a very large sequence that is initially almost sorted.

Summarising, we get that, if $N \leqslant 2^{d+1} - 2$, the algorithm requires a number of exchanges less than

$$(d-1) \cdot N + 2 \cdot d \approx (\log(N+2) - 2) \cdot (N+2)$$

and $2\frac{1}{2}$ times as many comparisons. An optimisation of part B can only replace the factor $2\frac{1}{2}$ by $2 + \varepsilon$ for some positive ε.

The standard measure of sortedness of an array a is the number of inversions, pairs $i < j$ with $a[i] > a[j]$. Since all swaps in our algorithm decrease the number of inversions, the number of swaps is bounded by the initial number of inversions. So, if the number of inversions is of order N, the number of swaps is also of order N. In that case, the main part of the complexity is due to part B of the algorithm.

3. Some statistics and a conclusion

We measured the performance of our algorithm and of some other in-situ sorting algorithms, viz. the versions of quicksort and heapsort given in [2], and of smoothsort, cf. [1]. We investigated arrays of sizes 100, 1000, 10000. For every array size, we considered the sorted array, an ascending array with 10% random values interspersed, and a completely random array. In Table 1 we give, for each of the four sorting algorithms and each of the nine sets of data, the number of exchanges divided by $N \cdot \log N$. Table

Table 1
Number of exchanges divided by $N \cdot \log N$

N	random %	quicksort	heapsort	smoothsort	poplar sort
100	0	0.09	0.96	0	0
	10	0.18	0.95	0.23	0.23
	100	0.29	0.86	0.79	0.57
1000	0	0.05	0.97	0	0
	10	0.13	0.97	0.33	0.19
	100	0.27	0.91	0.98	0.71
10000	0	0.04	0.99	0	0
	10	0.14	0.98	0.35	0.29
	100	0.26	0.93	1.08	0.79

Table 2
Number of comparisons divided by $N \cdot \log N$

N	random %	quicksort	heapsort	smoothsort	poplar sort
100	0	0.91	1.63	0.28	0.48
	10	1.00	1.62	0.68	0.80
	100	0.98	1.54	1.72	1.44
1000	0	0.90	1.76	0.20	0.50
	10	1.04	1.75	0.89	0.82
	100	1.04	1.69	2.16	1.77
10000	0	0.94	1.84	0.15	0.49
	10	0.97	1.82	0.85	0.98
	100	0.89	1.77	2.41	1.95

2 contains the number of comparisons divided by $N \cdot \log N$. Repeated experiments with other sets of random data yielded slightly different numerical results, but showed that these data are representative where a comparison of the algorithms is concerned.

Our conclusion is that we must acknowledge quicksort's superiority as a practical routine for a library. Since quicksort has a quadratic worst-case behaviour, however, poplar sort may be preferred for safety-critical real-time applications where the upper bound on execution time is important.

References

[1] E.W. Dijkstra, Smoothsort, an alternative for sorting in situ, *Sci. Comput. Programming* **1** (1982) 223–233, and Errata, *Sci. Comput. Programming* **2** (1982) 85.

[2] N. Wirth, *Algorithms + Data Structures = Programs* (Prentice-Hall, Englewood Cliffs, NJ, 1976).